

# Vanguard VDP: A Privacy-Preserving, Audit-Ready Vulnerability Disclosure Platform for Small Security Teams

**Version:** 1.0

**Date:** May 2026

**Author:** Francis Al Victoriano

**Repository:** <https://github.com/fr4nc1stein/vanguard>

**Live Platform:** <https://vanguard.laet4x.com>

---

## Abstract

Vulnerability Disclosure Programs (VDPs) have become a critical component of modern security posture, yet most small organizations are forced to choose between two inadequate options: an informal shared inbox with no audit trail, or expensive enterprise platforms designed for large bug bounty operations. Neither option adequately serves teams that need structured triage workflows, researcher accountability, and compliance-ready audit logs without the operational overhead of enterprise tooling.

This paper presents **Vanguard VDP**, an open-source, production-ready vulnerability disclosure platform built on a privacy-first, edge-native architecture. Vanguard VDP provides end-to-end encrypted report handling, cryptographically non-enumerable reference IDs, role-based access control, immutable audit logging with a per-actor scoped view model, a public researcher recognition system, and a triage workflow — all deployable to Cloudflare's global edge network at minimal cost.

We describe the platform's architecture, security model, auditability mechanisms, and researcher-facing trust features. We evaluate Vanguard VDP against common lightweight VDP workflows and identify its known limitations and planned extensions.

---

## 1. Problem Statement

Organizations operating security programs face a structural gap between informal and enterprise-grade VDP tooling.

### 1.1 The Shared Inbox Problem

The most common "VDP" for small teams is a dedicated email address ( `security@company.com` ). This approach has several compounding failure modes:

- **No structured triage state.** Reports move through email threads with no canonical status. It is impossible to answer "how many open reports do we have?" without reading every thread.
- **No role separation.** Anyone with mailbox access can read unencrypted vulnerability reports including researcher PII, exploit proof-of-concept details, and affected system information.
- **No audit trail.** There is no record of who read a report, when a status changed, or who made a triage decision.
- **No researcher accountability.** Anonymous or semi-anonymous researchers cannot be recognized, and there is no mechanism to track a researcher's history.
- **PII exposure risk.** Email archives containing researcher contact information and sensitive vulnerability details are often retained indefinitely without encryption.

## 1.2 The Enterprise Platform Gap

Commercial VDP platforms (HackerOne, Bugcrowd, Intigriti) solve these problems but introduce a different set of constraints:

- **Cost.** Enterprise licensing is prohibitive for small teams, startups, or open-source projects.
- **Operational overhead.** Platform configuration, policy management, and researcher onboarding require dedicated program management time.
- **Vendor dependency.** Sensitive vulnerability reports are stored in third-party infrastructure with limited control over encryption, retention, and data residency.
- **Feature mismatch.** Enterprise platforms are optimized for high-volume bug bounty programs, not the structured-but-lightweight workflows most small teams need.

## 1.3 What Small Teams Actually Need

A small security team running a VDP typically needs:

1. A structured submission form with severity classification and scope validation
2. A triage queue with status tracking and assignment
3. Role separation between researchers, triagers, and administrators
4. Encrypted storage of sensitive report content
5. An immutable audit trail for compliance and investigation
6. Researcher-facing transparency (acknowledgment, status updates, public recognition)
7. Deployment at low cost with minimal operational overhead

Vanguard VDP is designed to satisfy exactly this requirement set.

---

## 2. Platform Overview

Vanguard VDP is a full-stack web application that implements the complete lifecycle of a vulnerability disclosure report — from researcher submission through triage, resolution, and public recognition.

### 2.1 Core Modules

#### Researcher Submission ( `/submit` )

Authenticated researchers complete a structured submission form covering target scope, vulnerability type, severity classification (Critical/High/Medium/Low/Info), CVSS vector, title, and full description. The form enforces scope restrictions and allowed vulnerability types defined by administrators. All sensitive content is encrypted before reaching the database.

#### Researcher Dashboard ( `/dashboard` )

Researchers can track the status of their own submissions, view status history, and communicate with the triage team through a threaded comment system.

#### Triage Dashboard ( `/triage` )

Staff with TRIAGER or ADMIN roles access a paginated, filterable, and searchable report queue. Reports can be triaged individually with status changes, severity adjustments, assignment, and internal or external comments.

#### Report Detail and Communication ( `/triage/reports/[id]` )

Full report view for staff, including decrypted content, CVSS score, a two-way communication thread, and an audit log timeline showing all actions taken on the report.

**Scope Management ( /admin/scope )**

Administrators define the program's in-scope targets, allowed vulnerability types, severity restrictions, researcher-facing guidance, and exclusion paths. Submissions are validated server-side against active scope records.

**User Management ( /admin/users )**

Administrators manage researcher and triager accounts, including role promotion/demotion and account suspension. All role and suspension actions are audit-logged.

**Activity Logs ( /admin/activity-logs )**

A comprehensive, searchable, exportable audit log of all platform actions across all modules. Triagers have a scoped view limited to their own actions; administrators see all activity.

**Analytics Dashboard ( /admin/analytics )**

Summary statistics, severity and status distributions, submission trends over configurable time windows (7/30/90/365 days), top reporters, and CSV export.

**Hall of Fame ( /hall-of-fame )**

A public leaderboard and activity feed recognizing researchers whose reports have been accepted or resolved. Displays researcher rankings by points, time-period filters, and a Hacktivity feed of recent accepted reports with redacted titles.

**Response Templates ( /admin/templates )**

Administrators maintain a library of response templates for common triage communications, reducing response time and improving consistency.

---

**3. Architecture**

**3.1 Technology Stack**

| Layer          | Technology                      | Version |
|----------------|---------------------------------|---------|
| Framework      | Next.js App Router              | 16.2.5  |
| Runtime        | Cloudflare Workers via OpenNext | Latest  |
| Hosting        | Cloudflare Pages                | —       |
| Database       | Cloudflare D1 (SQLite)          | —       |
| ORM            | Drizzle ORM                     | 0.45.2  |
| Authentication | Clerk                           | 7.3.2   |
| Encryption     | Web Crypto API (AES-GCM-256)    | Native  |
| Validation     | Zod                             | 4.4.3   |
| Styling        | Tailwind CSS                    | v4      |
| Language       | TypeScript (strict mode)        | 5       |

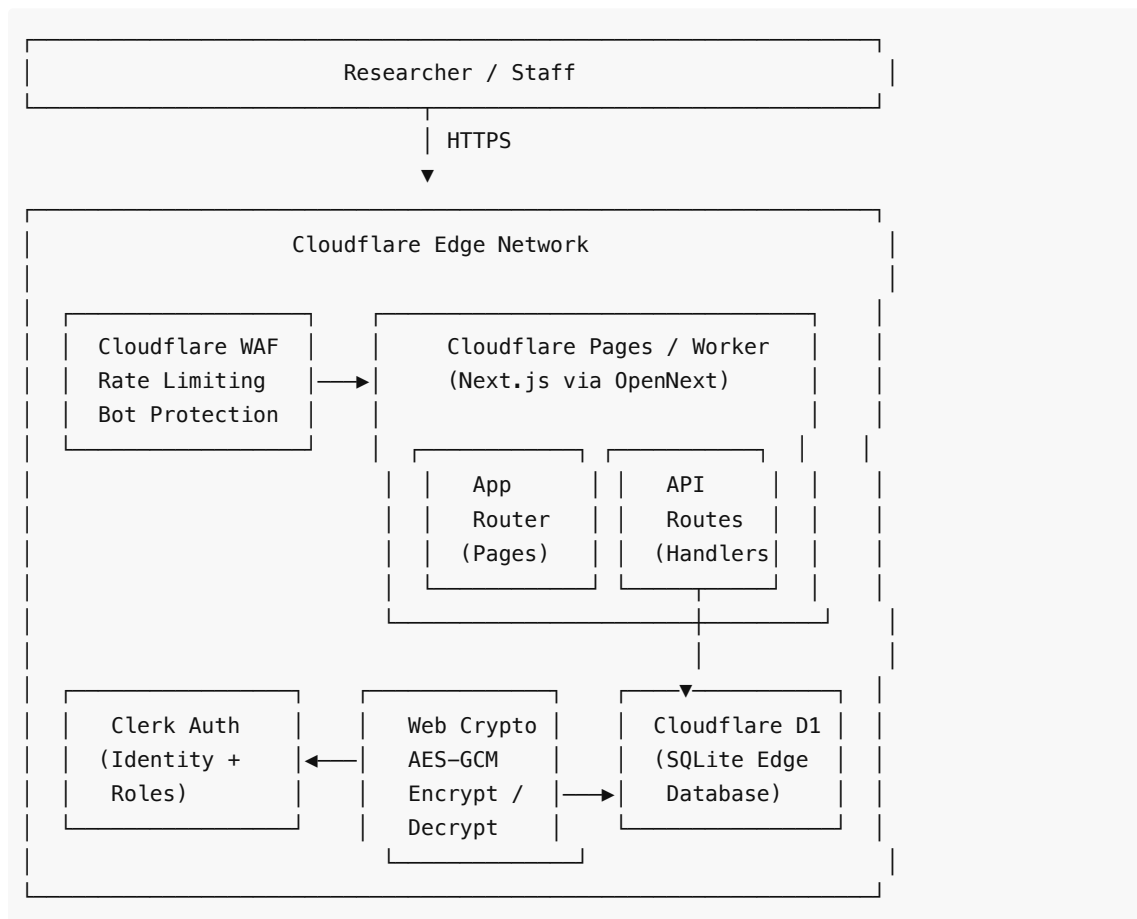
**3.2 Deployment Model**

Vanguard VDP runs entirely on Cloudflare's edge network. The Next.js application is compiled by `@opennextjs/cloudflare` and deployed to Cloudflare Pages, which distributes the application across Cloudflare's global edge locations. The database runs on Cloudflare D1, a serverless SQLite service with global read replication.

This architecture has several practical consequences:

- **No server to manage.** There are no VMs, containers, or orchestration systems to operate.
- **Global latency.** Requests are handled by the nearest Cloudflare edge location, typically within 50ms for most researchers globally.
- **Cost.** The platform runs within Cloudflare's free tier for most small programs, with no minimum spend.
- **Cold start elimination.** Cloudflare Workers do not have the cold start problem of serverless functions on other providers.

### 3.3 Architecture Diagram



### 3.4 Request Flow

A typical report submission follows this path:

1. Researcher authenticates via Clerk (OAuth or email/password)
2. The submission form performs client-side Zod validation (schema mirrored from server)
3. `POST /api/reports` receives the request; middleware confirms the session is valid
4. The API handler calls `requireRole('USER')` to verify authentication
5. Server-side Zod validation re-validates the payload independently

- 6. The report body and email are encrypted with AES-GCM-256 using a fresh 96-bit IV
- 7. The submitter IP is hashed (SHA-256) before storage — the plaintext is never persisted
- 8. A cryptographically secure reference ID is generated ( `VVDP-[S]-YYYY-XXXXXXX` )
- 9. The report is written to D1; an audit log entry is created in the same transaction
- 10. An optional Discord webhook fires with the reference ID and severity

3.5 Database Schema Summary

The core schema consists of 11 tables across two migrations epochs:

| Table              | Purpose                                      |
|--------------------|----------------------------------------------|
| reports            | Core report records with encrypted fields    |
| audit_logs         | Immutable action history with entity scoping |
| scopes             | In-scope targets with restriction rules      |
| comments           | Researcher-triager communication thread      |
| response_templates | Reusable triage response library             |
| hall_of_fame       | Public recognition entries                   |
| researcher_stats   | Aggregated researcher metrics                |
| hacktivity         | Public activity feed entries                 |
| points_config      | Configurable severity-to-points mapping      |

4. Security Model

4.1 Authentication

All protected routes require a valid Clerk session. Clerk handles credential storage, session management, multi-factor authentication, and OAuth integration. The platform does not store passwords.

Authentication is enforced at two layers:

- **Middleware ( `middleware.ts` ):** Protects page routes ( `/admin` , `/triage` , `/dashboard` , `/submit` ). Any unauthenticated request is redirected to `/sign-in` .
- **API route handlers:** Every protected API endpoint independently calls `requireRole()` , which fetches the caller's role from Clerk and throws a `403 Response` if the role is insufficient.

This two-layer design means that a bypass of middleware (e.g., a misconfigured route matcher) does not grant API access.

4.2 Role-Based Access Control

Roles are stored in Clerk's `publicMetadata.role` field. The platform defines three roles with a strict hierarchy:

| Role | Access Level                                                     |
|------|------------------------------------------------------------------|
| USER | Submit reports, view own dashboard, post comments on own reports |

|         |                                                                                                                  |
|---------|------------------------------------------------------------------------------------------------------------------|
| TRIAGER | All USER access + triage queue, report detail, status changes, internal comments, scoped audit log               |
| ADMIN   | All TRIAGER access + user management, scope management, full audit log, analytics, templates, Hall of Fame admin |

Role promotion and demotion are performed via the Clerk Backend API and are audit-logged. ADMIN role changes must be made directly in the Clerk dashboard to prevent accidental administrative escalation.

### 4.3 Encryption at Rest

All sensitive report content is encrypted using AES-GCM-256 before being written to D1:

- `reports.email_encrypted` — researcher contact email
- `reports.email_iv` — 96-bit IV for email decryption
- `reports.body_encrypted` — full report body (description, steps to reproduce, impact, evidence)
- `reports.body_iv` — 96-bit IV for body decryption

The encryption key is a 64-character hex string (32 bytes) stored as an environment variable ( `ENCRYPTION_KEY` ). Each encryption operation generates a fresh cryptographically random 96-bit IV via `crypto.getRandomValues()` . IVs are never reused.

Decryption is performed only in API handlers with `requireRole('TRIAGER')` or higher, and only when explicitly requested. Decryption events are audit-logged.

Report owners (the submitting researcher) can also decrypt their own report body to review their own submission. This access is logged separately from staff decryption.

### 4.4 PII Minimization

The platform is designed to minimize the collection and retention of personally identifiable information:

- **IP addresses are never stored in plaintext.** The submitter's IP is hashed with SHA-256 before storage. The hash supports rate-limiting lookups without enabling IP reconstruction.
- **Email addresses are encrypted.** Researcher contact details are encrypted at the application layer before reaching the database.
- **Researcher handles are optional.** Researchers may submit anonymously without providing a handle.
- **Console logging is PII-free.** Production logging uses only non-sensitive identifiers (reference ID, severity code, status) and is gated behind `NODE_ENV === 'development'` for verbose session data.
- **Public report titles are redacted.** Before appearing in the Hall of Fame or Hacktivity feed, report titles are automatically scanned and redacted for email addresses, IP addresses, API keys/tokens, long hex strings, and full URLs.

### 4.5 Reference ID Design

Each report is assigned a reference ID at submission time using the format `VVDP-[S]-YYYY-XXXXXXX` , where `[S]` is the severity code (C/H/M/L/I), `YYYY` is the submission year, and `XXXXXXX` is an 8-character uppercase hex string from `crypto.randomUUID()` .

This design provides:

- **Non-enumerability.** ~4.3 billion valid IDs per year per severity. Sequential scanning is infeasible.
- **Instant severity signal.** Triagers can identify report priority from the reference ID alone.

- **Immutability.** The severity encoded in the ID reflects the researcher's initial assessment and never changes, even if the triager adjusts severity during triage. The current severity is tracked separately in `reports.severity`.

## 4.6 SQL Injection Prevention

User-supplied search query parameters that feed into LIKE expressions are sanitized before use:

```
const sanitized = q.replace(/[%_\]/g, '\\$&');
conditions.push(like(reports.title, `%${sanitized}%`));
```

All other queries use Drizzle ORM's parameterized query builder, which prevents SQL injection by construction.

## 4.7 Rate Limiting

The platform implements two layers of rate limiting:

- **Application level:** Configurable per-endpoint rate limits applied in `lib/middleware/with-rate-limit.ts`. Report submission is limited to 3 requests per 60 seconds per client IP.
- **Cloudflare WAF:** Edge-level rate limiting rules that block or challenge requests before they reach the application. WAF rules also provide bot detection via Cloudflare Bot Fight Mode.

---

# 5. Auditability and Compliance

## 5.1 Audit Log Design

Every significant platform action produces an immutable record in the `audit_logs` table. The `logAudit()` helper is designed to never throw — audit failures are caught and logged without interrupting the primary action.

```
interface AuditLogEntry {
  id: string; // UUID
  reportId: string | null;
  entityType: 'report' | 'user' | 'system';
  entityId: string;
  actorId: string; // Clerk user ID
  actorEmail: string;
  action: AuditAction;
  oldValue: string | null;
  newValue: string | null;
  ipHash: string | null; // SHA-256(ip), never plaintext
  isInternal: 0 | 1;
  timestamp: number; // Unix ms
}
```

## 5.2 Action Taxonomy

The platform tracks 15 distinct action types covering all modules:

| Action | Module | Visibility |
|--------|--------|------------|
|--------|--------|------------|

|                  |           |          |
|------------------|-----------|----------|
| report_submitted | Reports   | Public   |
| status_changed   | Reports   | Public   |
| severity_changed | Reports   | Internal |
| assigned         | Reports   | Internal |
| report_viewed    | Reports   | Internal |
| report_decrypted | Reports   | Internal |
| poc_uploaded     | Reports   | Public   |
| comment_posted   | Comments  | Public   |
| template_used    | Templates | Internal |
| scope_created    | Scopes    | Internal |
| scope_updated    | Scopes    | Internal |
| scope_archived   | Scopes    | Internal |
| role_changed     | Users     | Internal |
| user_suspended   | Users     | Internal |
| user_unsuspended | Users     | Internal |

Internal actions are visible only to TRIAGER/ADMIN roles. Researchers see only public actions on their own reports.

### 5.3 Entity Scoping

The `audit_logs` table supports three entity types, allowing the audit log to capture events beyond individual reports:

- **report** — Actions on a specific vulnerability report (linked via `reportId` )
- **user** — Administrative actions on user accounts (role changes, suspensions)
- **system** — Platform-level events (scope creation, configuration changes)

This design prevents the `report_id` foreign key from being overloaded with non-report events, which was an early design deficiency corrected in migration `0011` .

### 5.4 Per-Triager Scoped Views

Triagers can access the activity log module at `/admin/activity-logs` , but their view is automatically scoped to their own `actor_id` . The scoping is enforced server-side in the API handler — the TRIAGER cannot bypass it by manipulating query parameters.

```
const { userId, role } = await requireRole('TRIAGER');
// Triagers are automatically scoped; admins get full access
const effectiveActorId = role === 'ADMIN' ? sp.get('actor_id') : userId;
```

The API response includes a `scoped: true` flag that the UI uses to display an informational banner: *"Showing your own actions only. Admins can view all platform activity."*

Administrators retain full filter access across all actors, entity types, action types, and date ranges.

### 5.5 CSV Export

Both the activity log view and the scoped export endpoint support CSV export for offline compliance review. The export respects all active filters (action type, actor, date range) and applies the same role-based scoping as the log view.

### 5.6 Report-Level Audit Timeline

Each report's detail page includes a visual audit timeline showing every action taken on that specific report, ordered chronologically. The timeline shows the actor's name (fetched dynamically from Clerk), the action, timestamp, and before/after values for status and severity changes. This provides a self-contained compliance record for any individual report.

---

## 6. Researcher Trust and Transparency

### 6.1 Hall of Fame and Public Recognition

Researchers whose reports are accepted or resolved are automatically recognized in the public Hall of Fame at `/hall-of-fame`. Recognition is awarded by a server-side trigger on status change — no manual administrative step is required.

The recognition system is points-based, with configurable point values per severity level:

| Severity | Default Points |
|----------|----------------|
| Critical | 1,000          |
| High     | 500            |
| Medium   | 250            |
| Low      | 100            |
| Info     | 50             |

The public leaderboard supports time-period filtering (All Time, This Month, This Year) and displays each researcher's severity breakdown alongside their total score.

### 6.2 Hacktivity Feed

The Hacktivity feed ( `/hall-of-fame` ) provides a chronological public activity feed of recently accepted and resolved reports. Each entry shows the researcher's display name, a redacted report title, severity, points awarded, and relative timestamp.

### 6.3 Title Redaction for Public Display

Report titles shown in public-facing features (Hall of Fame, Hacktivity) are automatically processed by a redaction function before storage. The redaction pass removes:

- Email addresses → `[EMAIL]`

- IPv4 addresses → [IP]
- API keys and tokens ( sk\_live\_ , pk\_live\_ , api\_key\_ , long hex strings) → [API\_KEY] / [TOKEN]
- Full URLs → domain only (paths and parameters stripped)
- Long file system paths → [PATH]

Administrators can override automatically redacted titles manually. The original encrypted title is preserved in the reports table and is never modified.

## 6.4 Two-Way Communication

Researchers and triagers communicate through a structured comment thread on each report. Comments can be marked as internal (staff-only) or public (visible to the researcher). Researchers see only their own reports and only public comments on those reports. Staff see all comments including internal notes.

Comment authorship is displayed by dynamically fetching names from Clerk, using a priority order: custom alias → full name → first name → username → email prefix → "Anonymous". User names are never stored in the database; they are always fetched fresh, which means the displayed name stays current if a researcher updates their Clerk profile.

## 6.5 Reference ID Transparency

Every researcher receives a structured reference ID immediately on submission ( VVDP-H-2026-7D4E2C10 ). The ID encodes the initial severity and submission year, giving researchers a permanent stable identifier to use in follow-up communications. The format is documented publicly and the ID is shown on the submission success page, in the researcher dashboard, and in all communications.

# 7. Evaluation

## 7.1 Comparison Against Common Alternatives

| Criterion              | Shared Inbox            | Vanguard VDP                  | Enterprise Platform |
|------------------------|-------------------------|-------------------------------|---------------------|
| Structured triage      | ✗ None                  | ✓ Full workflow               | ✓ Full workflow     |
| Role separation        | ✗ None                  | ✓ Three roles                 | ✓ Configurable      |
| Encrypted storage      | ✗ None                  | ✓ AES-GCM-256                 | ⚠ Vendor-dependent  |
| Audit log              | ✗ None                  | ✓ Immutable, exportable       | ✓ Varies            |
| PII minimization       | ✗ Full PII in plaintext | ✓ Hashed IPs, encrypted email | ⚠ Varies            |
| Researcher recognition | ✗ Manual                | ✓ Automatic                   | ✓ Full              |
| Deployment cost        | ~\$0                    | ~\$0 (Cloudflare free tier)   | \$\$\$\$            |
| Operational overhead   | Low                     | Low                           | High                |
| Data residency control | ✓ Own mail server       | ✓ Own CF account              | ✗ Vendor-hosted     |

|             |     |              |               |
|-------------|-----|--------------|---------------|
| Open source | N/A | ✅ Fully open | ❌ Proprietary |
| Edge-native | ❌   | ✅            | ⚠️ Varies     |

## 7.2 Security Properties

Vanguard VDP provides stronger security guarantees than informal workflows across three dimensions:

**Confidentiality.** Report content is encrypted before reaching the database. A database-level breach does not expose readable report content. Role-based access ensures only authorized staff can initiate decryption.

**Integrity.** The audit log is append-only with no update or delete operations exposed through the application layer. Every status transition, role change, and administrative action produces an immutable timestamped record.

**Accountability.** Every action in the system is attributed to an authenticated Clerk user ID. Actor identity is resolved to a display name at query time, not stored in the log, so the attribution remains accurate even if a user changes their name.

## 7.3 Known Limitations

**No formal retention policy controls.** The platform does not yet implement configurable log retention periods or automated purging. All audit logs accumulate indefinitely in D1. This is tracked as a dependency on the Program Settings module (VAN-11).

**No enterprise SSO or SAML.** Authentication is provided entirely by Clerk, which supports Google, GitHub, and email/password. SAML/LDAP enterprise SSO is not currently supported.

**Single-region database write path.** Cloudflare D1 provides global read replication but writes are routed to a primary region. For programs with globally distributed triager teams, write latency from distant regions may be perceptible.

**No advanced workflow automation.** There are no SLA timers, escalation rules, or automated triager assignment. Status progression is entirely manual.

**Limited multi-program support.** The platform is designed for a single disclosure program. Running multiple independent programs with separate scope, roles, and reporting would require separate deployments.

**Clerk dependency for identity.** User identity, role storage, and session management are fully delegated to Clerk. Organizations with strict vendor requirements for identity providers should evaluate this dependency.

---

## 8. Future Work

The following capabilities are planned in the project roadmap:

**Program Settings (VAN-11).** A configurable settings module for SLA windows, bounty ranges, notification rules, and log retention policies.

**Retention Policy Controls (VAN-16, deferred).** Configurable log retention periods and automated purging, dependent on the Program Settings module.

**Notification System (VAN-19).** Email notifications for status changes, in-app notification centre, and configurable webhook integrations beyond Discord.

**Researcher Experience (VAN-20).** Report draft saving, automatic duplicate detection, guided submission wizard, and researcher-facing vulnerability templates.

**Triage Enhancements (VAN-21).** Bulk status actions, custom report labels, saved filter sets, CVE linking, and bounty tracking.

**Public Researcher API (VAN-22).** A documented public API with API key authentication allowing researchers to submit and track reports programmatically.

**Integration Management (VAN-12).** An admin UI for managing Slack, Jira, and custom webhook integrations, replacing the current hardcoded Discord webhook environment variable.

**Dashboard Enhancements (VAN-23).** Real-time triage queue updates via server-sent events, dark mode, and keyboard shortcuts.

**Advanced Analytics (VAN-15).** Custom date range pickers, scheduled report delivery, and a more comprehensive trend analysis view.

---

## 9. Conclusion

Vanguard VDP demonstrates that a production-grade vulnerability disclosure platform — with encrypted report handling, role-based access control, immutable audit logging, and researcher recognition — can be built and operated at minimal cost using modern edge-native infrastructure.

The platform fills a genuine gap in the current ecosystem: it is more structured and secure than a shared inbox, more operationally lightweight than enterprise VDP tooling, and fully open-source with no vendor lock-in beyond the underlying infrastructure services.

By choosing Cloudflare D1 and Workers over traditional database servers and containers, and by delegating identity to Clerk rather than building authentication from scratch, the platform achieves enterprise-grade security properties without the engineering investment those properties typically require.

Vanguard VDP is appropriate for: startups building their first security program, open-source projects accepting vulnerability reports, small security teams at organizations not yet ready for enterprise VDP investment, and as a reference architecture for privacy-preserving security tooling on edge infrastructure.

The platform is production-deployed at <https://vanguard.laet4x.com> and the full source code is available at <https://github.com/fr4nc1stein/vanguard>.

---

## Appendix A: API Surface Reference

### Public Endpoints (No Authentication Required)

| Method | Path                    | Description                   |
|--------|-------------------------|-------------------------------|
| GET    | /api/scopes             | List active in-scope targets  |
| GET    | /api/hall-of-fame       | Public researcher leaderboard |
| GET    | /api/hall-of-fame/stats | Aggregate program statistics  |
| GET    | /api/hacktivity         | Recent public activity feed   |

### Researcher Endpoints (Authentication Required)

| Method | Path                       | Description                   |
|--------|----------------------------|-------------------------------|
| POST   | /api/reports               | Submit a vulnerability report |
| GET    | /api/reports               | List own reports              |
| GET    | /api/reports/[id]          | Get own report detail         |
| GET    | /api/reports/[id]/comments | List comments on own report   |
| POST   | /api/reports/[id]/comments | Post a comment                |

**Staff Endpoints (TRIAGER or ADMIN)**

| Method | Path                            | Description                        |
|--------|---------------------------------|------------------------------------|
| GET    | /api/admin/reports              | Paginated report list with filters |
| GET    | /api/admin/reports/[id]         | Full report detail with decryption |
| PATCH  | /api/admin/reports/[id]/status  | Triage action                      |
| GET    | /api/admin/stats                | Dashboard statistics               |
| GET    | /api/admin/analytics            | Analytics data                     |
| GET    | /api/admin/activity-logs        | Activity log (role-scoped)         |
| GET    | /api/admin/activity-logs/export | CSV export (role-scoped)           |

**Admin Endpoints (ADMIN Only)**

| Method         | Path                          | Description                 |
|----------------|-------------------------------|-----------------------------|
| GET/POST       | /api/admin/scopes             | Manage scope targets        |
| PATCH/DELETE   | /api/admin/scopes/[id]        | Update or archive scope     |
| GET/PATCH      | /api/admin/users              | List users, update roles    |
| POST           | /api/admin/users/[id]/suspend | Suspend or unsuspend user   |
| GET/POST/PATCH | /api/admin/templates          | Manage response templates   |
| GET/PATCH      | /api/admin/hall-of-fame/*     | Manage Hall of Fame entries |

---

**Appendix B: Environment Variables**

| Variable         | Required | Description                                            |
|------------------|----------|--------------------------------------------------------|
| ENCRYPTION_KEY   | Yes      | 64-char hex (32 bytes). Generate: openssl rand -hex 32 |
| CLERK_SECRET_KEY | Yes      | Clerk backend API key                                  |

|                                   |     |                                 |
|-----------------------------------|-----|---------------------------------|
| NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY | Yes | Clerk frontend publishable key  |
| DISCORD_WEBHOOK_URL               | No  | New report notification webhook |

---

## Appendix C: Deployment Commands

```
# Local development (no D1 binding)
npm run dev

# Local development with full Cloudflare stack (D1 available)
npm run dev:cf

# Build for Cloudflare Pages
npm run pages:build

# Deploy to Cloudflare Pages
npm run deploy

# Apply a migration to local D1
npx wrangler d1 execute vanguard-security --local --file=migrations/0001_schema.sql

# Apply a migration to production D1
npx wrangler d1 execute vanguard-security --remote --file=migrations/0001_schema.sql

# Generate a new ENCRYPTION_KEY
openssl rand -hex 32
```

---

Vanguard VDP is released under the MIT License.

Repository: <https://github.com/fr4nc1stein/vanguard>